



Department of Computer Science and Engineering
University of Puerto Rico
Mayagüez Campus

INSO 6006 – Database Systems Engineering Fall 2026

Project #1: The Slotted Heap File and Its Free-Space Lists
Due Date: October 23, 2026, 11:59 PM

Objectives

- Understand slotted-page storage of fixed-width records and the (page, slot) addressing of rows in a heap file.
- Understand a free-space index maintained as two header-addressed doubly-linked lists whose membership is derived from occupancy.
- Gain experience with the pin/unpin protocol of a buffer manager, including multi-page list surgery under a strict pin discipline.
- Understand how an on-disk data structure verifies itself, and how a check suite uses that verification.

Overview

You will complete the storage layer of a small single-table SQL database engine. The supplied project contains the block-file layer, the buffer pool and buffer manager, the slotted-page code, the tuple codec and JSON catalog, the SQL front end, the binder and executors, the check suite, and the build configuration.

You will receive the project directory with the member method bodies of **heapfile/HeapFile.cpp** removed — every declaration stays in **heapfile/HeapFile.h**, which documents the full contract. Your task is to implement those method bodies until the check suite passes.

Do not modify the provided header files or the interfaces of supplied components.

The engine stores one table as a heap file: a sequence of 4096-byte blocks whose first block is a header page and whose remaining blocks are slotted data pages. The header addresses two doubly-linked lists of data pages — pages with free space, and full pages — so INSERT picks its

page in constant time instead of walking a chain. Your implementations must cooperate with the existing code and pass the supplied checks.

The heap file format

Block 0 is the header page; blocks 1, 2, ... are data pages. A page's id IS its block number, so page 0 is the header and data pages start at 1. Every page, header included, is a slotted page: the last 12 bytes of the block are a trailer holding the slot count and the two page links (prev, next), and the bytes before them split into slots of (record + occupancy byte).

The record width is fixed by the table's schema: 4 bytes per Integer column, 8 per Double, and the catalog-declared width per String column.

The header page's slots are never used. Its record area carries the identification record, laid out as:

offset	size	field
0	4	magic "TUP2"
4	4	free_head first page with a free slot (kNoPage = empty)
8	4	free_tail last page appended to the free list
12	4	full_head first page whose every slot is used
16	4	full_tail last page of the full list
20	4	next_page_id the id the next allocation will use (starts 1)
24	4	name_length
28	...	table name bytes

The layout constants, the HeaderFields helpers, and the identification record writers/checkers for this format are already provided inside HeapFile.cpp and fix the on-disk format; implement your member methods on top of them and do not change them.

The two free-space lists

The data pages are organized as TWO doubly-linked lists addressed by the header, using each page's trailer links as the list links:

- The FREE list: every data page that has at least one free slot;
- The FULL list: every data page with every slot occupied.
-

Your implementation must maintain these invariants:

1. Every allocated data page (ids 1 through next_page_id - 1) sits on exactly one list. Page 0 is on no list.
2. Membership is DERIVED, never stored: a page belongs on the full list if and only if every one of its slots is occupied. There is no membership bit — a stored bit could drift silently, while the derived rule turns every drift into a check_structure failure.
3. Both lists are appended pages at the TAIL, so each list's order reflects allocation (and re-allocation) order: bulk loads scan in insertion order, and a page emptied by erases is reused oldest-first.

4. Pages are allocated once and never removed. A page whose records were all erased stays on the free list; the file never shrinks, and `next_page_id` only grows.

Lifecycle: create, open, close

1. **create** formats a one-block file: refuse an open `HeapFile` and a path that already holds a non-empty file (an existing `EMPTY` path is formatted in place); validate that the record size can fit a page; allocate page 0; format it as a slotted page; write the identification record; then close (flushing) and leave the `HeapFile` closed. Users will `open()` it to use it later on when needed.
2. **open** validates an existing file: probe that the path exists (the manager's open would otherwise create one), register the file with the buffer manager, then check block 0 — identification, slotted-page attach, and the header's list pointers in range. On any failure unwind completely (close through the manager, reset the handle) and return false; on success bind the schema, the serializer, and the record size, and mark the file open.
3. **close** flushes and closes only this file, and resets the `HeapFile`'s state only when the manager actually released the file, so a failed close (a page still pinned) can be retried. After a successful close every later operation fails with "heap file is not open".

The pin protocol

Every disk access goes through the injected buffer manager. Plain row operations hold exactly ONE pinned page at a time: pin, act, unpin. List movement (a page joining or moving between lists) transiently holds up to FIVE pins — the moving page plus the header, the page's two list neighbors, and the destination list's tail. The discipline:

- 1 Acquire ALL auxiliary pins before the first write, in ascending page-id order. The set is always distinct, so the order is well-defined.
- 2 Once all pins are held, the surgery is pure memory writes — it cannot fail mid-way, so a membership change is either fully applied or not attempted
- 3 Allocation happens only after the pins it needs are held.

The session's pool of 8 frames is comfortable for this; a pool of 2 can fail where 8 succeeds. Your implementation must honor the discipline so the small pools work.

Row operations

- 1 **insert**: serialize the tuple (validating it first — serialization failures change nothing and pin no page); take the free list's `HEAD` page as the target — or, when the free list is empty, allocate a fresh page (id taken from `next_page_id`, which then advances) and append it to the free list; store the record in the target's first free slot; and move the page to the full list when its last free slot was taken. Report the new row id as a (page, slot) pair.

- 2 **find**: pin the record page, copy the record out, unpin, and deserialize into the tuple. Fails with a named error when the row id names the header page, a page beyond the file's extent, an out-of-range slot, a free slot, or a record that violates the schema.
- 3 **update**: serialize, pin the record page, overwrite the record in place, and unpin. Records are all the same size and occupancy stays 1, so an update never changes how full the page is and NEVER moves the page between lists.
- 4 **erase**: pin the record page, note whether the page WAS full, clear the slot's occupancy byte (the record bytes stay behind as garbage), and move a just-unfilled page back to the TAIL of the free list — so the oldest-emptied page is reused first.
- 5 **pin_record_page**: the shared path for find, update, and erase — reject the header page and pages beyond the file's extent (resident pages count as existing even before write-back), pin the page, attach a slotted-page view, and require the slot occupied. The caller owns the pin on success; the helper releases the pin on failure.
- 6 **List-surgery primitives**: **append_fresh_page** makes the just-allocated page the free list's only element (head and tail), clears the page's own links, and advances `next_page_id` past it; it is only called right after an allocation, when the free list is empty by construction.
- 7 **move_between_lists** detaches the pinned page from its source list (its links name the source neighbors) and appends it at the destination list's tail, under one pin acquisition as described above, so the transition never leaves the page on no list or on both.

The scan

scan() pins the header once, reads the two list heads and `next_page_id`, unpins, and returns an iterator seeded with them. The iterator then walks the free list's pages, then the full list's pages, in list order; on each page it scans slots in order and yields every occupied slot's decoded record, remembering the slot to resume from. Once both lists are exhausted, every further `next()` call keeps reporting exhaustion: false with an EMPTY error — callers distinguish end-of-scan from failure by testing the string. A revisited page means a cyclic list and is a named failure, never a silent loop.

The structural self-check

check_structure walks both lists from the header, one pin at a time, and fails with a named error when any of these holds: a head or tail pointer that disagrees with the walk (a list is empty exactly when both of its ends are `kNoPage`); a cycle, an unreachable page, or a page visited twice; a page on a list contradicting its occupancy; the header page appearing on a list; the union of the two lists not covering exactly the allocated page ids (1 through `next_page_id - 1`); or an on-disk data page the walk cannot reach. A dirty page may not be on disk yet, so the reverse is not an error. An empty file (header only) passes. The check is a diagnostic: the storage operations themselves never call it, but the check suite runs it after every storage operation of the storage tests.

Conventions

Every method that can fail takes a trailing **std::string& error**, clears it on entry, and fills it with a named message exactly when it returns false. Never throw. On failure the file's contents are unchanged — transitions are either-or, never half-applied — and every pin taken on a failure path is released without disturbing the error already carried.

Tasks for Project 2

You will receive the Git repository with the member method bodies of one implementation file removed. Complete and implement this file:

```
heapfile/HeapFile.cpp
```

All header files (heapfile/HeapFile.h, heapfile/SlottedPage.h), the provided helpers inside HeapFile.cpp (the header-page layout constants, the HeaderFields read/write helpers, the identification record writers and checkers, and the free function `erase_file`), the slotted-page code, the buffer manager and buffer pool, the block-file code, the replacement policy, the catalog and tuple serializer, the SQL front end, the binder and executors, the test programs, and the build configuration are provided. Do not modify any of them.

Testing and Submission

Build the project with CMake and run the supplied check executable or CTest suite:

```
cmake -S . -B build
cmake --build build
ctest --test-dir build --output-on-failure
# or directly:
./build/p2stdbeng
```

The checks name the behavior they verify. The storage checks run `check_structure` after every operation — including a randomized 200-operation insert/erase sequence cross-checked against a shadow model — and hand-built cases cover the list-surgery corner cases, so a relink mistake surfaces at the step that caused it. Test lifecycle failures and recovery, page placement and allocation, fill and unfill list moves, reuse of erased pages before new allocations, scan order and exactly-once coverage, update leaving list membership alone, corruption detection, and small buffer pools.

Submit only heapfile/HeapFile.cpp unless your instructor specifies a different submission procedure.

PROJECT DUE DATE: [SET DUE DATE], 11:59 PM